Sesión 6

FUNDAMENTOS DE REDES NEURONALES PARA NLP

De la neurona biológica a los modelos profundos

Por

Carlos César Sánchez Coronel

2026

Índice general

Índice general **1**

1. Fundamentos de Redes Neuronales para NLP **2**
1.1. Logro de la sesión . . . 2
1.2. Introducción: ¿Por qué redes neuronales en NLP? . . . 2
1.3. La neurona artificial: el bloque fundamental . . . 3
1.3.1. Analogía biológica . . . 3
1.3.2. Modelo matemático . . . 3
1.3.3. Funciones de activación comunes . . . 3
1.4. Redes feed-forward (perceptrón multicapa) . . . 4
1.4.1. Arquitectura general . . . 4
1.4.2. Propagación hacia adelante (forward pass) . . . 4
1.4.3. Ejemplo: clasificación binaria con una capa oculta . . . 5
1.5. Entrenamiento de una red neuronal . . . 5
1.5.1. Función de pérdida (loss function) . . . 5
1.5.2. Retropropagación (backpropagation) . . . 5
1.5.3. Descenso por gradiente (gradient descent) . . . 6
1.6. Conexión con NLP: de palabras a vectores . . . 6
1.6.1. Representación de texto para una red neuronal . . . 6
1.6.2. Capa de embedding . . . 6
1.6.3. Arquitectura para clasificación de texto con embeddings + MLP . . . 6
1.7. Panorama de arquitecturas neuronales en NLP . . . 7
1.7.1. Redes feed-forward (perceptrón multicapa) — *el punto de partida* . . . 7
1.7.2. Redes convolucionales (CNN) — *detectores de patrones locales* . . . 8
1.7.3. Redes recurrentes (RNN) — *modelado de secuencias* . . . 8
1.7.4. Transformers — *el paradigma actual* . . . 10
1.7.5. Resumen comparativo . . . 10
1.7.6. ¿Dónde nos encontramos y hacia dónde vamos? . . . 10
1.8. Ejemplo completo en PyTorch . . . 11
1.8.1. Preparación de datos . . . 11
1.8.2. Definición del modelo . . . 11
1.8.3. Entrenamiento . . . 12
1.8.4. Evaluación . . . 12
1.9. Regularización y buenas prácticas . . . 13
1.10. Casos de uso en la industria . . . 13
1.11. Resumen y conexión con el resto del curso . . . 13

Capítulo 1

Fundamentos de Redes Neuronales para NLP

En la sesión anterior trabajamos con *word embeddings* estáticos (Word2Vec, GloVe) y modelos clásicos de machine learning. Sin embargo, el verdadero potencial del aprendizaje profundo surge cuando integramos estos embeddings en arquitecturas neuronales que pueden **aprender de forma conjunta** la representación de las palabras y la tarea final. Esta sesión sienta las bases de las redes neuronales artificiales: desde el modelo de una neurona hasta las redes feed-forward, explicando los conceptos matemáticos y mostrando su aplicación en problemas de Procesamiento de Lenguaje Natural (NLP).

1.1. Logro de la sesión

Comprender los principios fundamentales de las redes neuronales (neurona artificial, funciones de activación, propagación hacia adelante, retropropagación) y ser capaz de construir una red simple para clasificación de texto utilizando PyTorch, asentando así los cimientos para arquitecturas más avanzadas (RNN, LSTM, Transformers).

1.2. Introducción: ¿Por qué redes neuronales en NLP?

Los modelos clásicos (regresión logística, SVM, etc.) requieren características diseñadas manualmente o representaciones fijas del texto. Las redes neuronales permiten:

- **Aprendizaje de representaciones**: la propia red aprende qué características (embeddings) son útiles para la tarea.
- **No linealidad**: pueden modelar relaciones complejas entre palabras.
- **Arquitecturas flexibles**: desde simples MLP hasta complejos Transformers.

En NLP, esto se traduce en mejoras significativas en tareas como análisis de sentimiento, traducción automática, respuesta a preguntas, etc.

1.3. La neurona artificial: el bloque fundamental

1.3.1. Analogía biológica

Una neurona biológica recibe señales a través de las dendritas, las procesa en el cuerpo celular y, si se supera un umbral, envía una señal a través del axón. La neurona artificial (perceptrón) imita este comportamiento:

- Entradas $x_1, x_2, \ldots, x_n$ (señales de entrada).
- Pesos sinápticos $w_1, w_2, \ldots, w_n$ (importancia de cada entrada).
- Sesgo b (umbral).
- Función de activación f (decisión de disparo).

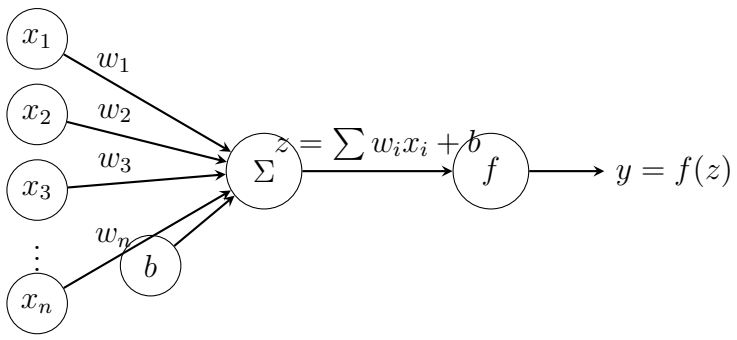

Figura 1.1: Modelo de una neurona artificial.

1.3.2. Modelo matemático

La salida de una neurona se calcula en dos pasos:

1. Combinación lineal de las entradas:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

2. Aplicación de una función de activación no lineal:

$$y = f(z)$$

1.3.3. Funciones de activación comunes

- **Sigmoide**: $\sigma(z) = \frac{1}{1+e^{-z}}$. Útil para probabilidades (salida entre 0 y 1). Sufre de saturación (gradientes pequeños).
- **Tangente hiperbólica (tanh)**: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$. Salida entre -1 y 1, centrada en cero.
- **ReLU (Rectified Linear Unit)**: $\text{ReLU}(z) = \text{máx}(0, z)$. Muy usada en capas ocultas, evita saturación para valores positivos.

- **Softmax**: generalización de la sigmoide para múltiples clases. Convierte un vector de logits en probabilidades:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

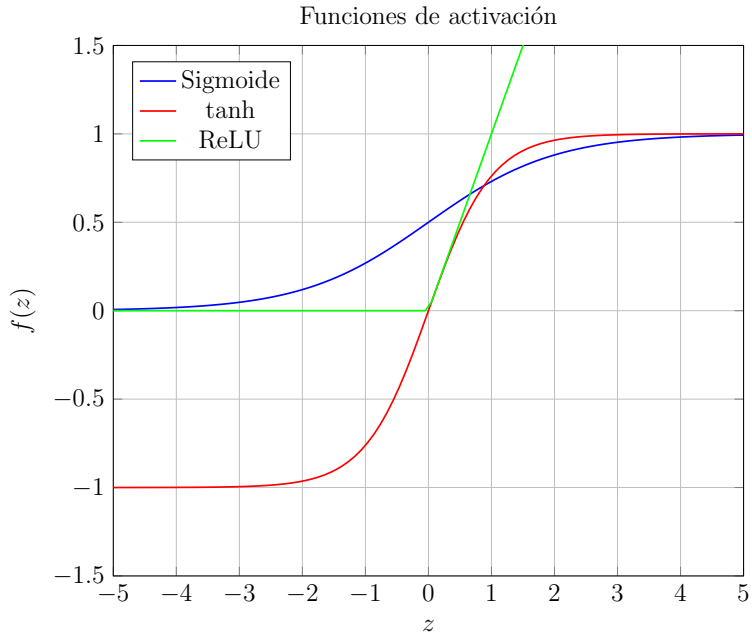

Figura 1.2: Comparación de funciones de activación.

1.4. Redes feed-forward (perceptrón multicapa)

Una red feed-forward está formada por varias capas de neuronas:

- **Capa de entrada**: representa los datos (por ejemplo, índices de palabras o embeddings).
- **Capas ocultas**: transformaciones no lineales.
- **Capa de salida**: produce la predicción final.

1.4.1. Arquitectura general

1.4.2. Propagación hacia adelante (forward pass)

Para una red con L capas, la salida de la capa l se calcula como:

$$\mathbf{a}^{(l)} = f^{(l)}\left(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\right)$$

donde $\mathbf{a}^{(0)} = \mathbf{x}$ (entrada), $\mathbf{W}^{(l)}$ y $\mathbf{b}^{(l)}$ son los pesos y sesgos de la capa l, y $f^{(l)}$ es la función de activación.

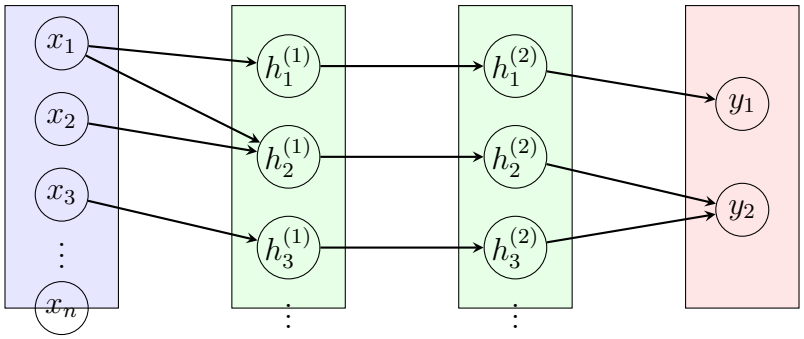

Figura 1.3: Red feed-forward con dos capas ocultas.

1.4.3. Ejemplo: clasificación binaria con una capa oculta

Para una red con una capa oculta de tamaño H y activación ReLU, y capa de salida con sigmoide:

$$\mathbf{h} = \mathrm{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \quad \mathbf{h} \in \mathbb{R}^H$$
$$\hat{y} = \sigma(\mathbf{w}^{(2)} \cdot \mathbf{h} + b^{(2)}), \quad \hat{y} \in (0, 1)$$

1.5. Entrenamiento de una red neuronal

1.5.1. Función de pérdida (loss function)

Mide el error entre la predicción $\hat{y}$ y el valor real y. Para clasificación, la más común es la **entropía cruzada** (cross-entropy). Para un problema con C clases:

$$\mathcal{L} = -\sum_{c=1}^{C} y_c \log(\hat{y}_c)$$

donde y_c es 1 si la clase es la correcta, 0 en otro caso (one-hot), y $\hat{y}_c$ es la probabilidad predicha por softmax.

1.5.2. Retropropagación (backpropagation)

Es el algoritmo que calcula el gradiente de la pérdida con respecto a todos los parámetros de la red, aplicando la regla de la cadena. Intuitivamente, se propaga el error desde la salida hacia atrás, actualizando los pesos para minimizar la pérdida.

$$\text{Entrada } \mathbf{x} \xleftarrow{\frac{\partial \mathcal{L}}{\partial \mathbf{x}}} \text{Pesos } \mathbf{W} \xleftrightarrow[\frac{\partial \mathcal{L}}{\partial \mathbf{W}}]{} z = \mathbf{W}\mathbf{x} + b \xleftrightarrow[\frac{\partial \mathcal{L}}{\partial z}]{} a = f(z) \xleftrightarrow[\frac{\partial \mathcal{L}}{\partial a}]{} \text{Pérdida } \mathcal{L}$$

Figura 1.4: Flujo de la retropropagación. Las flechas sólidas representan el forward pass; las punteadas, el backward pass.

1.5.3. Descenso por gradiente (gradient descent)

Una vez calculados los gradientes, se actualizan los parámetros en la dirección opuesta al gradiente:

$$\mathbf{W}_{\text{nuevo}} = \mathbf{W}_{\text{anterior}} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}}$$

donde η es la tasa de aprendizaje (learning rate).

1.6. Conexión con NLP: de palabras a vectores

1.6.1. Representación de texto para una red neuronal

Para que una red neuronal procese texto, debemos convertir las palabras en números. El enfoque moderno es usar **capas de embedding** (ya vistas en la sesión anterior).

1.6.2. Capa de embedding

Una capa de embedding es una tabla de búsqueda: asigna a cada palabra (índice) un vector denso entrenable. Si el vocabulario tiene V palabras y la dimensión del embedding es d, la capa tiene una matriz $\mathbf{E} \in \mathbb{R}^{V \times d}$. Dada una secuencia de índices $[i_1, i_2, \ldots, i_T]$, la capa devuelve los vectores correspondientes:

$$\mathbf{e}_t = \mathbf{E}[i_t], \quad \mathbf{e}_t \in \mathbb{R}^d$$

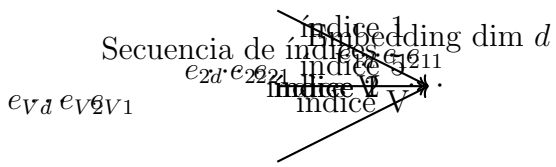

Figura 1.5: Funcionamiento de una capa de embedding.

1.6.3. Arquitectura para clasificación de texto con embeddings + MLP

La forma más simple de usar embeddings en una red es:

1. Obtener los embeddings de todas las palabras de la oración.
2. Combinarlos en un único vector (por ejemplo, promediándolos).
3. Pasar ese vector por un MLP para obtener la clasificación.

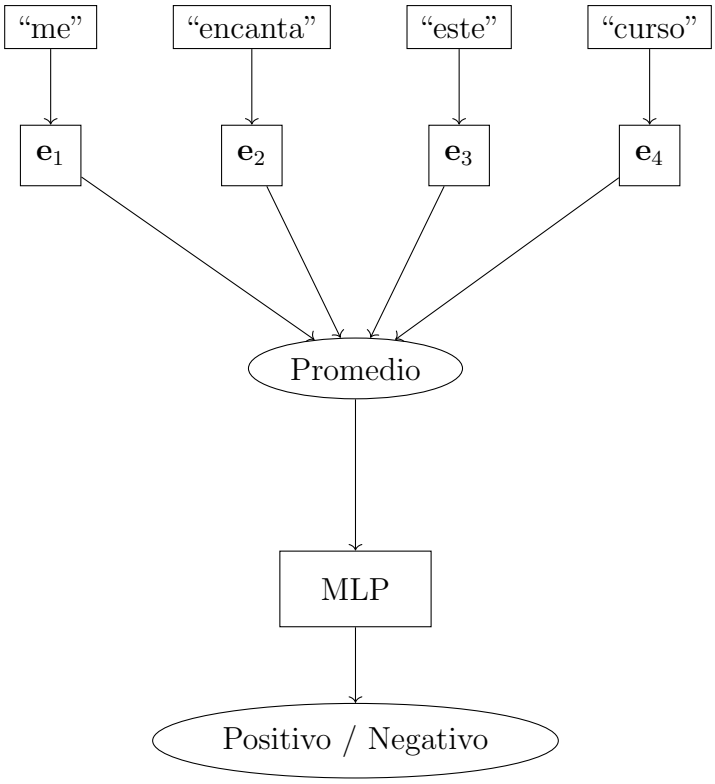

Figura 1.6: Arquitectura de clasificación de texto con embeddings y MLP.

1.7. Panorama de arquitecturas neuronales en NLP

Antes de adentrarnos en las redes recurrentes y los transformadores en sesiones posteriores, es fundamental contar con una visión global de los distintos tipos de redes neuronales utilizadas en Procesamiento de Lenguaje Natural. Cada arquitectura aborda el problema desde una perspectiva diferente: algunas modelan el orden de las palabras, otras capturan patrones locales, y otras procesan la secuencia completa en paralelo. Entender sus fortalezas y debilidades te permitirá elegir la más adecuada según la tarea, los recursos y la naturaleza de los datos.

A continuación, presentamos una clasificación por **arquitectura**, que es la forma más común de organizar estos modelos. Dentro de cada tipo, se explican sus capacidades, limitaciones y casos de uso típicos.

1.7.1. Redes feed-forward (perceptrón multicapa) — *el punto de partida*

- **¿Qué hacen?** Procesan entradas de tamaño fijo (por ejemplo, un vector que representa una oración completa mediante el promedio de embeddings) y aplican transformaciones no lineales a través de capas densas (fully connected). No consideran el orden de las palabras.

- **¿Qué NO hacen?** No modelan relaciones secuenciales ni dependencias contextuales. La entrada se trata como un conjunto desordenado (bolsa de palabras).

- **Cuándo se usan en NLP:** Como línea base en clasificación de textos cortos, análisis de sentimiento simple o tareas donde el orden no es crucial (por ejemplo,

clasificación de temas). Son rápidas, fáciles de entrenar y requieren pocos datos.

- **Ejemplo típico:** El clasificador desarrollado en esta sesión, que promedia embeddings y los pasa por un MLP.
- **Lugar en la historia:** Fueron las primeras redes aplicadas a NLP (años 90 y 2000), pero hoy se consideran un bloque fundamental, no el estado del arte.

1.7.2. Redes convolucionales (CNN) — *detectores de patrones locales*

- **Origen:** Las CNN se popularizaron en visión por computadora para procesar imágenes, donde aplican filtros que detectan bordes, texturas, etc. En NLP, se adaptaron para operar sobre secuencias de embeddings de palabras.
- **¿Qué hacen?** Aplican filtros (kernels) que se deslizan sobre la secuencia de vectores (palabras) para detectar patrones locales, como n-gramas relevantes (por ejemplo, "muy buena", "no me gusta"). Luego, combinan la información mediante operaciones de pooling (max-pooling, average-pooling) para obtener una representación fija de la oración.
- **¿Qué NO hacen?** No modelan dependencias de largo alcance de forma explícita; el contexto se limita al tamaño del filtro. Tampoco capturan bien el orden secuencial más allá de la ventana local. Además, al igual que las MLP, la entrada debe tener longitud fija (o se usa padding).
- **Cuándo se usan en NLP:** Clasificación de textos donde ciertas frases clave son determinantes (ej. detección de spam, análisis de sentimiento con expresiones fijas). También en modelos híbridos (CNN + RNN) o para extraer características de nivel de palabra antes de una RNN.
- **Ejemplo típico:** El modelo de Kim (2014) que usa CNN sobre word2vec para clasificación de oraciones.
- **Lugar en la historia:** Populares entre 2014 y 2017, hoy en día han sido superadas por Transformers, pero siguen siendo eficientes en entornos con recursos limitados y en tareas donde los patrones locales son suficientes.

1.7.3. Redes recurrentes (RNN) — *modelado de secuencias*

Las RNN están diseñadas específicamente para procesar datos secuenciales, como texto, audio o series temporales. Mantienen un estado oculto que se actualiza a medida que recorren la secuencia, lo que les permite capturar dependencias temporales y el orden de las palabras. Dentro de esta familia, encontramos varias variantes que mejoran la capacidad de memoria y estabilidad del entrenamiento.

RNN simple (Elman RNN)

- **¿Qué hace?** Procesa la secuencia paso a paso. En cada instante t t, actualiza su estado oculto h t h t en función de la entrada actual x t x t y el estado anterior h t 1 h t1 : h t = tanh (W x h x t + W h h h t 1 + b h) h t =tanh(W xh x t

+W hh h t1 +b h). Puede usarse para producir una salida en cada paso o una salida global al final.

- **¿Qué NO hace?** Sufre del problema de desvanecimiento (vanishing gradient) y explosión del gradiente, lo que impide aprender dependencias de largo plazo (más allá de 10-15 pasos). Además, su naturaleza secuencial impide la paralelización.
- **Cuándo se usa:** Tareas con dependencias a corto plazo, como etiquetado gramatical (POS tagging) o generación de texto simple, aunque hoy se prefieren variantes más avanzadas.

LSTM (Long Short-Term Memory)

- **¿Qué hace?** Introduce un mecanismo de compuertas (input, forget, output) que controlan el flujo de información, permitiendo mantener o descartar información a lo largo de muchos pasos. Esto resuelve en gran medida el problema del desvanecimiento del gradiente.
- **¿Qué NO hace?** Sigue siendo secuencial (difícil de paralelizar) y tiene más parámetros que una RNN simple, lo que aumenta el costo computacional y el riesgo de sobreajuste si los datos son escasos.
- **Cuándo se usa:** Traducción automática, generación de texto, reconocimiento de voz, y cualquier tarea con dependencias de largo plazo. Hasta 2017, era el estado del arte.
- **Ejemplo típico:** Modelo seq2seq con LSTM para traducción automática (Sutskever et al., 2014).

GRU (Gated Recurrent Unit)

- **¿Qué hace?** Es una variante más simple que LSTM, con solo dos compuertas (reset y update). Tiene menos parámetros y suele ser más rápida de entrenar, con un rendimiento comparable en muchas tareas.
- **¿Qué NO hace?** Al igual que LSTM, mantiene la limitación de procesamiento secuencial y puede no ser tan expresiva como LSTM en ciertos problemas.
- **Cuándo se usa:** Similar a LSTM, pero preferida cuando se busca eficiencia computacional o se tienen menos datos.

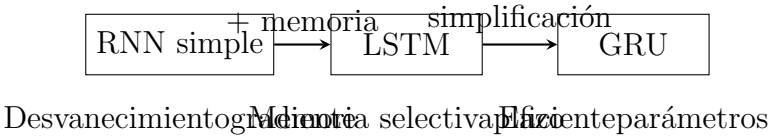

Figura 1.7: Evolución de las redes recurrentes: de RNN simple a LSTM y GRU.

1.7.4. Transformers — *el paradigma actual*

- **¿Qué hacen?** Procesan toda la secuencia en paralelo mediante un mecanismo de *self-attention*. En lugar de recorrer paso a paso, calculan la relevancia de cada palabra respecto a todas las demás, generando representaciones contextuales ricas. Esto permite capturar dependencias globales sin limitaciones de distancia y con alta paralelización.
- **¿Qué NO hacen?** Son computacionalmente costosos (la atención tiene complejidad cuadrática con la longitud de la secuencia) y requieren grandes cantidades de datos para entrenar desde cero. Sin embargo, el *transfer learning* (modelos pre-entrenados como BERT, GPT) ha democratizado su uso.
- **Cuándo se usan:** Prácticamente todas las tareas modernas de NLP: clasificación, generación, preguntas-respuestas, resumen, etc. Si tienes acceso a un modelo pre-entrenado, es la mejor opción.
- **Ejemplo típico:** BERT para clasificación de texto, GPT para generación.
- **Lugar en la historia:** Desde 2017 (Vaswani et al., "Attention is All You Need") son el estado del arte indiscutible.

1.7.5. Resumen comparativo

La siguiente tabla resume las capacidades clave de cada arquitectura, ayudándote a decidir cuál usar según la tarea:

Arquitectura	**Orden**	**Dependen**
No	No	Al
No	Alta	Mode
Baja	Bajos	Etiquetado de secuenci
Altos	Traducción, generación Transformers	S
Estado del arte en casi todo height		

Cuadro 1.1: Comparación de arquitecturas neuronales en NLP.

1.7.6. ¿Dónde nos encontramos y hacia dónde vamos?

En esta sesión hemos construido una red **feed-forward con embeddings**. Es el cimiento: hemos aprendido a representar texto como vectores y a pasarlo por capas densas. Este modelo no captura orden ni dependencias, pero nos sirve como punto de partida y como baseline.

En las próximas sesiones:

- **Sesión 7:** Redes recurrentes (RNN, LSTM, GRU) — modelaremos el orden y las dependencias secuenciales.
- **Sesión 8:** Transformers y atención — el estado del arte, procesamiento paralelo y modelos pre-entrenados (BERT, GPT).

1.8. Ejemplo completo en PyTorch

1.8.1. Preparación de datos

```
import torch
import torch.nn as nn
import torch.optim as optim
from collections import Counter

# Datos de ejemplo
texts = [
    "I loved this movie, it was fantastic",
    "Terrible film, I hated it",
    "Amazing acting and great story",
    "Boring and slow, waste of time"
]
labels = [1, 0, 1, 0]  # 1: positivo, 0: negativo

# Construir vocabulario
def build_vocab(texts, max_size=10000):
    word_counts = Counter()
    for text in texts:
        word_counts.update(text.lower().split())
    vocab = {word: idx+2 for idx, (word, _) in enumerate(
       word_counts.most_common(max_size))}
    vocab['<PAD>'] = 0
    vocab['<UNK>'] = 1
    return vocab

vocab = build_vocab(texts)
vocab_size = len(vocab)
print("Vocabulario:", vocab)

# Tokenizar y codificar
def encode(text, vocab, max_len=10):
    tokens = text.lower().split()
    indices = [vocab.get(token, vocab['<UNK>']) for token in
       tokens[:max_len]]
    indices += [vocab['<PAD>']] * (max_len - len(indices))
    return indices

X = [encode(text, vocab) for text in texts]
X = torch.tensor(X)  # (4, 10)
y = torch.tensor(labels)
```

1.8.2. Definición del modelo

```
class TextClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=50, hidden_dim=32,
       num_classes=2, dropout=0.5):
```

```
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
           padding_idx=0)
        self.fc1 = nn.Linear(embed_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, num_classes)
        self.dropout = nn.Dropout(dropout)
        self.relu = nn.ReLU()

    def forward(self, x):
        # x: (batch_size, seq_len)
        embedded = self.embedding(x)          # (batch, seq_len,
           embed_dim)
        pooled = embedded.mean(dim=1)         # (batch, embed_dim
           )
        h = self.dropout(self.relu(self.fc1(pooled)))  # (batch,
           hidden_dim)
        out = self.fc2(h)                     # (batch,
           num_classes)
        return out

model = TextClassifier(vocab_size)
```

1.8.3. Entrenamiento

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01, weight_decay
   =1e-5)

epochs = 100
for epoch in range(epochs):
    optimizer.zero_grad()
    logits = model(X)
    loss = criterion(logits, y)
    loss.backward()
    optimizer.step()

    if (epoch+1) % 20 == 0:
        print(f'Epoch {epoch+1}, Loss: {loss.item():.4f}')
```

1.8.4. Evaluación

```
with torch.no_grad():
    logits = model(X)
    predictions = torch.argmax(logits, dim=1)
    accuracy = (predictions == y).float().mean()
    print(f'Accuracy: {accuracy:.2%}')
```

1.9. Regularización y buenas prácticas

- **Dropout**: apaga aleatoriamente neuronas durante el entrenamiento para evitar co-adaptación.
- **Weight decay (L2)**: penaliza pesos grandes.
- **Early stopping**: detiene el entrenamiento cuando la pérdida en validación deja de mejorar.
- **Normalización por lotes (BatchNorm)** o **normalización de capas (LayerNorm)**: estabilizan el entrenamiento.
- **División de datos**: entrenamiento, validación, prueba.

1.10. Casos de uso en la industria

- **Análisis de sentimiento en redes sociales**: clasificar tweets como positivos, negativos o neutros.
- **Detección de spam**: identificar correos no deseados.
- **Clasificación de intenciones en chatbots**: saber si el usuario quiere comprar, reclamar o preguntar.
- **Moderación de contenido**: detectar comentarios tóxicos en foros.
- **Categorización de tickets de soporte**: asignar automáticamente a los departamentos correspondientes.

1.11. Resumen y conexión con el resto del curso

- Hemos visto los fundamentos de las redes neuronales: neurona, activaciones, MLP, forward y backward.
- Aplicamos estos conceptos a NLP mediante capas de embedding y una arquitectura simple de clasificación.
- Este conocimiento es la base para entender arquitecturas más avanzadas:
  - **RNN/LSTM** (próxima sesión): procesan secuencias respetando el orden.
  - **Transformers**: atención y procesamiento paralelo.
- Dominar estos conceptos es esencial para trabajar con modelos modernos como BERT, GPT, etc.